

VIP Шпаргалка: Трансформеры и большие языковые модели

Афшин Амиди и Шервин Амиди
Переведено: Богатырёв Умар

24 мая 2026 г.

В этой VIP шпаргалке находится обзор того, что содержится в книге “Super Study Guide: Transformers & Large Language Models”, которая включает в себя около ~600 иллюстраций на 250 страницах и подробно описывает следующие концепции. Более подробную информацию вы можете найти на странице: <https://superstudy.guide>.

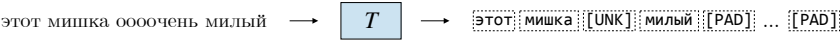
1 Основы

1.1 Токены

Определение – Токен (маркер) - это неделимая единица текста, такая как слово, подслово или символ, и является частью предопределённого словаря.

Примечание: Неизвестный маркер [UNK] представляет неизвестные фрагменты текста, в то время как маркер заполнения [PAD] используется для заполнения пустых позиций, чтобы обеспечить согласованную длину последовательности ввода.

Токенизатор – Токенизатор T делит текст на токены произвольного уровня детализации.



Вот основные типы токенизаторов:

Тип	Плюсы	Минусы	Пример
Слово	• Легко интерпретируется • Короткая последовательность	• Большой словарный запас • Варианты слов не обрабатываются	плюшевый мишка
Подслово	• Использование корней слов • Интуитивно понятный эмбединг	• Увеличена длина последовательности • Токенизация стала более сложной	плюш: ##евый: мишка
Байтовый символ	• Отсутствие проблем с нехваткой словарного запаса • Небольшой словарный запас	• Гораздо большая длина последовательности • Паттерны трудно интерпретировать из-за слишком низкого уровня	п:л:ю:ш:е:в ы:й:м:и:ш к:a:

Примечание: Кодирование пар байтов (Byte-Pair Encoding, BPE) и Униграмма (Unigram) являются широко используемыми токенизаторами уровня подслов.

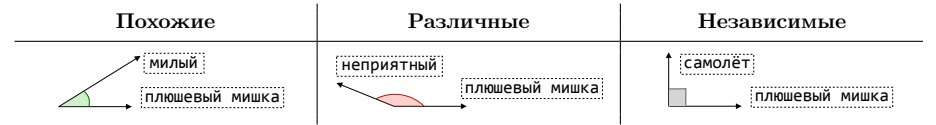
1.2 Эмбединги

Определение – Эмбединги - это числовое представление элемента (например, токена, предложения), которое характеризуется вектором $x \in \mathbb{R}^n$.

Сходство – Косинусное сходство между двумя токенами t_1, t_2 количественно определяется с помощью:

similarity(t_1, t_2) = $\frac{t_1 \cdot t_2}{||t_1|| ||t_2||} = \cos(\theta) \in [-1, 1]$

Угол θ характеризует сходство между двумя токенами:

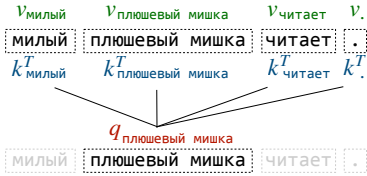


Примечание: Поиск приблизительно ближайших соседей (Approximate Nearest Neighbors, ANN) и локально-чувствительное хеширование (Locality Sensitive Hashing, LSH) — это методы, которые эффективно аппроксимируют операцию сходства в больших базах данных.

2 Трансформеры

2.1 Внимние

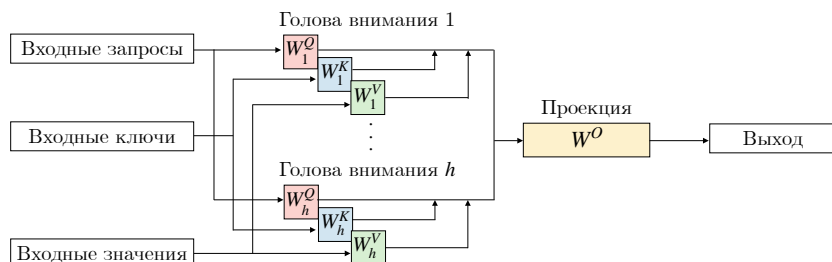
Формула – Учитывая запрос q , мы хотим знать, на какой ключ k запрос должен обратить “внимание” в отношении соответствующего значения v .



Внимание может быть эффективно вычислено с использованием матриц Q, K, V , которые содержат запросы q , ключи k и значения v соответственно, а также размерность d_k ключей:

attention = softmax $\left(\frac{QK^T}{\sqrt{d_k}} \right) V$

□ **Многоголовый механизм внимания** – Слой многоголового механизма внимания (*Multi-Head Attention*, МНА) выполняет вычисления внимания в нескольких головах, а затем проецирует результат в выходное пространство.

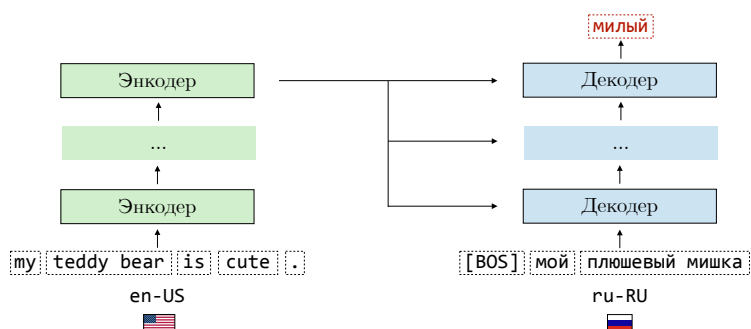


Он состоит из h голов, а также матриц W^Q, W^K, W^V , которые проецируют входные данные для получения запросов Q , ключей K и значений V . Проекция выполняется с использованием матрицы W^O .

Примечание: Механизм внимания с группировкой запросов (Grouped-Query Attention, GQA) и механизм внимания с несколькими запросами (Multi-Query Attention, MQA) — это разновидности МНА, которые сокращают вычислительные затраты за счёт совместного использования ключей и значений между разными головами внимания.

2.2 Архитектура

□ **Обзор** – Трансформер – это знаковая модель, основанная на механизме самовнимания и состоящая из энкодеров и декодеров. Энкодеры вычисляют осмысленные эмбединги входных данных, которые затем используются декодерами для предсказания следующего токена в последовательности.

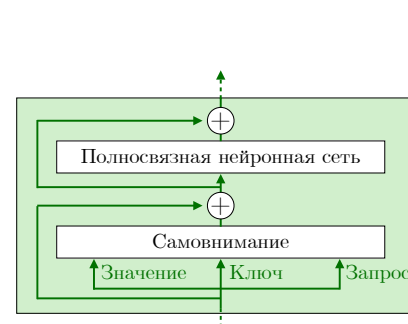


Примечание: Хотя изначально Трансформер был предложен в качестве модели для задач перевода, в настоящее время он широко используется во многих других приложениях.

□ **Компоненты** – Энкодер (encoder) и декодер (decoder) являются двумя основными компонентами Трансформера и выполняют разные функции:

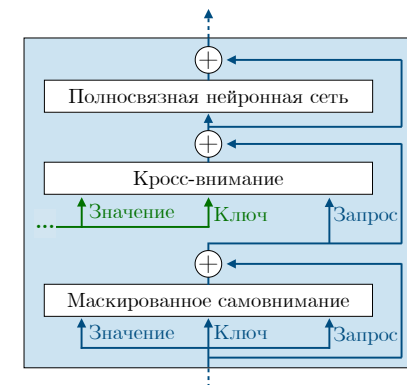
Энкодер

Закодированные эмбединги содержат (инкапсулируют) смысл входных данных



Декодер

Декодированные эмбединги содержат смысл как входных данных, так и выходных данных, предсказанных до этого момента

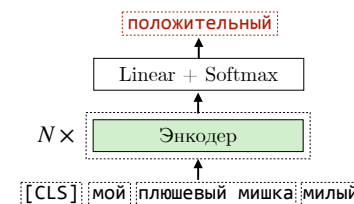


□ **Позиционные эмбединги** – Позиционные эмбединги сообщают о том, где находится токен в предложении, и имеют ту же размерность, что и эмбединги токенов. Они могут быть либо произвольно заданы, либо обучены на данных.

Примечание: Ротационные позиционные эмбединги (Rotary Position Embeddings, RoPE) — это популярная и эффективная вариация, которая вращает векторы запросов (query) и ключей (key) для учета относительной позиции токенов.

2.3 Варианты

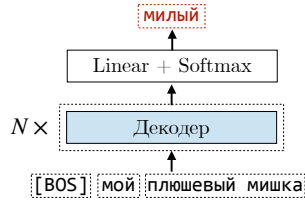
□ **Архитектура только с энкодером** – BERT (*Bidirectional Encoder Representations from Transformers*) — это модель на основе Трансформера, состоящая из стека энкодеров, которая принимает на вход текст и выдает осмысленные эмбединги, которые затем могут использоваться для последующих задач классификации.



В начале последовательности добавляется токен [CLS], чтобы передать смысл предложения. Его закодированный эмбединг часто используется в последующих (downstream) задачах, таких как определение тональности.

□ **Архитектура только с декодером** – Модель GPT (*Generative Pre-trained Transformer*) — это авторегрессивная модель на основе Трансформера, состоящая из стека декодеров. В

отличие от BERT и его производных, GPT рассматривает все задачи как задачи “текст в текст”.



Большинство современных LLM, таких как серия GPT, LLaMA, Mistral, Gemma, DeepSeek и др., основаны на архитектуре только с декодером.

Примечание: Модели энкодер-декодер (encoder-decoder), такие как T5, также являются авторегрессивными и имеют много общих характеристик с моделями, работающими только с декодером.

2.4 Оптимизация

□ **Аппроксимация внимания** – Вычисления механизма внимания имеют сложность $\mathcal{O}(n^2)$, что может быть дорогостоящим при увеличении длины последовательности n . Существует два основных метода для приближения этих вычислений:

- **Разреженность (Sparsity)**: механизм самовнимания (self-attention) не применяется ко всей последовательности, а только между более релевантными токенами.



- **Низкоранговость (Low-rank)**: формула внимания упрощается как произведение низкоранговых матриц, что снижает вычислительную нагрузку.

□ **Flash-внимание** – Flash-внимание — это точный метод, который оптимизирует вычисления внимания за счёт эффективного использования GPU, применяя быструю статическую оперативную память (Static Random-Access Memory, SRAM) для операций с матрицами перед записью результатов в более медленную память с высокой пропускной способностью (High Bandwidth Memory, HBM).

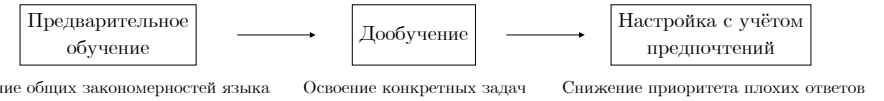
Примечание: На практике это сокращает использование памяти и ускоряет вычисления.

3 Большие языковые модели

3.1 Обзор

□ **Определение** – Большая языковая модель (Large Language Model, LLM) - это модель на основе Трансформера с широкими возможностями NLP. Она “большая” в том смысле, что обычно содержит миллиарды параметров.

□ **Жизненный цикл** – Обучение LLM проходит в три этапа: предварительное обучение, дообучение и настройка с учётом предпочтений.

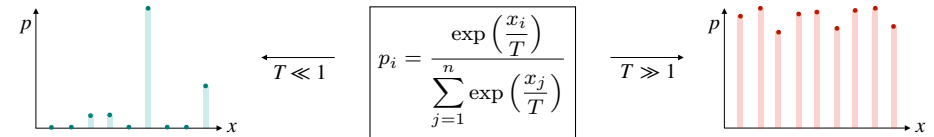


Дообучение и настройка с учётом предпочтений — это подходы после основного обучения, направленные на то, чтобы модель выполняла определённые задачи.

3.2 Подсказка

□ **Длина контекста** – Длина контекста модели - это максимальное количество токенов, которое может поместиться во входные данные. Обычно оно составляет от десятков тысяч до миллионов токенов.

□ **Сэмплирование при декодировании** – Предсказания токенов выбираются из предсказанного распределения вероятностей p_i , которое контролируется гиперпараметром температуры T .



Примечание: Высокие температуры приводят к более творческим результатам, в то время как низкие температуры приводят к более детерминированным результатам.

□ **Цепочка рассуждений** – Цепочка рассуждений (Chain-of-Thought, CoT) - это процесс рассуждения, в ходе которого модель разбивает сложную проблему на ряд промежуточных шагов. Это помогает модели генерировать правильный окончательный ответ. Дерево мыслей (Tree of Thoughts, ToT) - это более продвинутая версия CoT.

Примечание: Самосогласованность (Self-consistency) - это метод, который объединяет ответы по различным путям рассуждения.

3.3 Дообучение

□ **Контролируемое дообучение** – Контролируемое дообучение (Supervised Fine-Tuning, SFT) — это подход после основного обучения, который настраивает поведение модели в соответствии с конкретной задачей. Он опирается на высококачественные пары “вход-выход”, согласованные с задачей.

Примечание: Если данные SFT относятся к инструкциям, то этот шаг называется “настройка инструкций”.

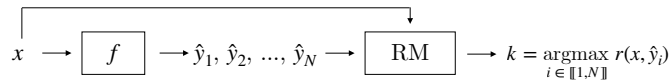
□ **Параметрически эффективное дообучение** – Параметрически эффективное дообучение (Parameter-Efficient Fine-Tuning, PEFT) — это категория методов, позволяющих эффективно выполнять контролируемое дообучение (SFT). В частности, низкоранговая адаптация (Low-Rank Adaptation, LoRA) аппроксимирует обучаемые веса W , фиксируя W_0 и обучая вместо этого низкоранговые матрицы A и B :

$$\begin{matrix} k \\ \hline \boxed{W} \\ \hline d \end{matrix} \approx \begin{matrix} k \\ \hline \boxed{W_0} \\ \hline d \end{matrix} + \begin{matrix} r \\ \hline \boxed{B} \\ \hline d \end{matrix} \times \begin{matrix} r & k \\ \hline \boxed{A} \\ \hline r \end{matrix}$$

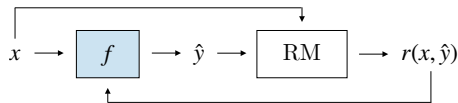
Примечание: Другие методы PEFT включают настройку префикса и вставку слоя адаптера.

3.4 Настройка с учётом предпочтений

□ **Модель вознаграждения** – Модель вознаграждения (*Reward Model*, RM) — это модель, которая предсказывает, насколько хорошо вывод $\hat{y}$ соответствует желаемому поведению при данном входе x . Метод Best-of-N (*Best-of-N*, BoN), также называемый методом отбора с отклонением (*rejection sampling*), использует модель вознаграждения для выбора наилучшего ответа среди N сгенерированных вариантов.



□ **Обучение с подкреплением** – Обучение с подкреплением (*Reinforcement Learning*, RL) — это подход, который использует модель вознаграждения (RM) и обновляет модель f на основе вознаграждений за её сгенерированные ответы. Если модель вознаграждения основана на человеческих предпочтениях, этот процесс называется обучением с подкреплением на основе обратной связи человека (*Reinforcement Learning from Human Feedback*, RLHF).

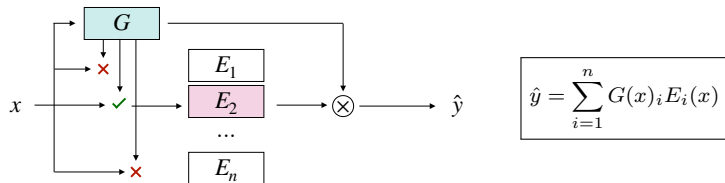


Оптимизация проксимальной политики (*Proximal Policy Optimization*, PPO) — это популярный алгоритм обучения с подкреплением (RL), который стимулирует получение более высоких вознаграждений, одновременно удерживая модель близкой к базовой модели, чтобы предотвратить “обман” вознаграждений.

Примечание: Существуют также контролируемые подходы, такие как прямая оптимизация предпочтений (Direct Preference Optimization, DPO), которые объединяют модель вознаграждения (RM) и RL в одном шаге контролируемого обучения.

3.5 Оптимизация

□ **Группа экспертов** – Группа экспертов (*Mixture of Experts*, MoE) — это модель, которая активирует только часть своих нейронов во время инференса. Она основана на “шлюзе” (*gate*) G и экспертах $E_1, \dots, E_n$.



LLM на основе MoE используют этот механизм “шлюза” в своих FFNN-слоях.

Примечание: Обучение LLM на основе MoE известно своей сложностью, как отмечается в статье про LLaMA, авторы которой решили не использовать эту архитектуру, несмотря на её эффективность во время инференса.

□ **Дистилляция** – Дистилляция — это процесс, при котором (небольшая) студентская модель S обучается на предсказаниях (большой) учительской модели T . Обучение проводится с использованием функции потерь KL-дивергенции:

$$KL(\hat{y}_T || \hat{y}_S) = \sum_i \hat{y}_T^{(i)} \log \left(\frac{\hat{y}_T^{(i)}}{\hat{y}_S^{(i)}} \right)$$

Примечание: Метки для обучения рассматриваются как “мягкие” (soft) метки, поскольку они представляют собой вероятности классов.

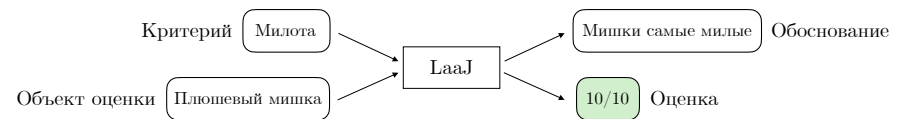
□ **Квантирование** – Квантирование модели - это категория методов, которые снижают точность весовых коэффициентов модели, но при этом ограничивают влияние этого снижения на итоговую производительность модели. В результате уменьшается объём памяти модели и ускоряется её инференс.

Примечание: QLoRA - это широко используемый квантованный вариант LoRA.

4 Приложения

4.1 LLM в роли судьи

□ **Определение** – LLM в роли судьи (*LLM-as-a-Judge*, LaaJ) — это метод, который использует LLM для оценки заданных выходов согласно определённым критериям. Важно, что он также способен генерировать обоснование своей оценки, что повышает интерпретируемость.



В отличие от метрик до эпохи LLM, таких как метрика (*Recall-Oriented Understudy for Gisting Evaluation*, ROUGE), LaaJ не требует эталонного текста, что делает его удобным для оценки любых типов задач. В частности, LaaJ демонстрирует сильную корреляцию с человеческими оценками, когда он опирается на крупную мощную модель (например, GPT-4), поскольку для успешной работы требуется способность к рассуждению.

Примечание: LaaJ полезен для проведения быстрых циклов оценок, но важно отслеживать соответствие между результатами LaaJ и оценками персонала, чтобы убедиться в отсутствии расхождений.

□ **Типичные систематические отклонения в поведении модели** – Модели LaaJ могут демонстрировать следующие отклонения:

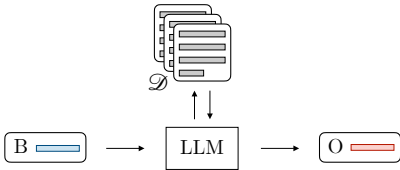
	Смещение позиции	Смещение из-за многословия	Смещение в сторону самопохвалы
Проблема	Предпочтение отдается первой позиции в парных сравнениях	Предпочитает более подробный контент	Благоприятствует результатам, генерируемым ими самими
Решение	Средний показатель по рандомизированным позициям	Добавьте штраф к выходной длине	Используйте судью, созданного на основе другой базовой модели

Решением этих проблем может быть точная настройка пользовательского LLaJ, но это требует больших усилий.

Примечание: Приведённый выше список отклонений не является исчерпывающим.

4.2 Генерация с использованием поиска информации

❑ **Определение** – Генерация с использованием поиска информации (*Retrieval-Augmented Generation*, RAG) — это метод, который позволяет LLM получать доступ к релевантным внешним источникам знаний для ответа на заданный вопрос. Это особенно полезно, если требуется использовать информацию, появившуюся после даты окончания предобучения LLM.



При наличии базы знаний $\mathcal{Q}$ и вопроса, Ретривер выбирает наиболее релевантные документы, затем дополняет запрос соответствующей информацией, прежде чем генерировать выходные данные.

Примечание: Этап извлечения обычно опирается на эмбединги, полученные с помощью моделей только энкодеров (*encoder-only*).

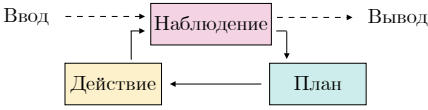
❑ **Гиперпараметры** – База знаний $\mathcal{Q}$ инициализируется путём разбиения документов на фрагменты размером n_c и преобразования их в векторы размерности $\mathbb{R}^d$.



4.3 Агенты

❑ **Определение** – Агент - это система, которая автономно преследует цели и выполняет задачи от имени пользователя. Для этого она может использовать различные цепочки вызовов LLM.

❑ **Размышляй и действуй** – Размышляй и действуй (*Reason + Act*, ReAct) — это фреймворк, который позволяет использовать несколько цепочек вызовов LLM для выполнения сложных задач:



Этот фреймворк состоит из следующих шагов:

- *Наблюдение (Observe)*: Синтезируйте предыдущие действия и явно укажите, что известно на данный момент.
- *Планирование (Plan)*: Подробно опишите, какие задачи необходимо выполнить и какие инструменты использовать.
- *Действие (Act)*: Выполните действие через API или найдите релевантную информацию в базе знаний.

Примечание: Оценка агентной системы является сложной задачей. Тем не менее, это можно делать как на уровне компонентов через локальные входы и выходы, так и на уровне системы через цепочки вызовов.

4.4 Модель рассуждений

❑ **Определение** – Модель рассуждений — это модель, которая опирается на цепочки рассуждений (*Chain-of-Thought*, CoT) для решения более сложных задач в математике, программировании и логике. Примеры моделей рассуждений включают серию o от OpenAI, DeepSeek-R1 и Google Gemini Flash Thinking.

Примечание: DeepSeek-R1 явно выводит свою цепочку рассуждений между тегами *<think>*.

❑ **Масштабирование** – Для расширения возможностей рассуждения используются два типа методов масштабирования:

	Описание	Иллюстрация
Масштабирование на этапе обучения	Продолжайте обучение с подкреплением (RL) дольше, чтобы модель научилась генерировать цепочки рассуждений в стиле CoT перед тем, как дать ответ	
Масштабирование на этапе инференса	Позвольте модели дольше “размышлять” перед тем, как дать ответ, используя ключевые слова для управления временем, такие как “Wait”	